

## Code

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

dataset = pd.read_csv('dataset.csv')

dataset

dataset.isnull().sum()

dataset.describe()

dummy = pd.get_dummies(dataset['Gender'])

dummy.head()

df=pd.concat((dataset,dummy), axis = 1)

df.head()

df=df.drop(['Gender'],axis=1)

df

df=df.drop(['female'],axis=1)

df.head()

first_column=df.pop('male')

df.insert(0, 'male', first_column)

df.head()
```

```
df.rename(columns={'male':'Gender'})

X = df.iloc[:,0:9].values

print(X)

y = df.iloc[:, -1].values

print(y)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=0)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

df.isnull().sum()

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

y_train_predict = model.predict(X_train)

print(y_train_predict)

from sklearn.metrics import

mean_absolute_error, mean_squared_error, explained_variance_score, r2_score

y_true = [3, -0.5, 2, 7]

y_train_pred = [2.5, 0.0, 2, 8]

r2_score(y_true, y_train_pred)

print(r2_score(y_train, y_train_predict))

print(mean_absolute_error(y_train, y_train_predict))
```

```

import numpy as np

print(np.sqrt(mean_squared_error(y_train,y_train_predict)))

print(explained_variance_score(y_train, y_train_predict))

ytest_pred = model.predict(X_test)

print(ytest_pred)

print(r2_score(y_test, ytest_pred))

#plotting the observed and the predicted remifentanil flow

import matplotlib.pyplot as plt

#setting the boundaries and the parameters

plt.rcParams['figure.figsize']=(16,6)

x_ax = range(len(X_test))

#plotting

plt.plot(x_ax, y_test,label ='Observed', color ='k', linestyle ='-')

plt.plot(x_ax,ytest_pred,label = 'Predicted', color ='k', linestyle ='--')

plt.ylabel('Remifentanil Flow')

plt.xlabel('Testing sample data')

plt.legend(bbox_to_anchor = (0.5,-0.2),loc = 'lower center', ncol =2, frameon = False)

plt.show()

!pip install yellowbrick

from yellowbrick.regressor import PredictionError

visualizer = PredictionError(model)

```

```
visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

from yellowbrick.regressor import ResidualsPlot

visualizer = ResidualsPlot(model)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

visualizer.predict([[0,65.0,33,114,80,95,74,88,1]])

visualizer.predict([[1,74.5,35,112,75,110,62,80,1]])

visualizer.predict([[0,68.6,46,147,92,103,75,74,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,1]])

visualizer.predict([[1,100,76,154,110,88,90,87,1]])

visualizer.predict([[0,80,56,134,90,68,70,67,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,2]])

visualizer.predict([[1,80,56,134,90,68,70,67,3]])

visualizer.predict([[1,80,56,134,90,68,70,67,0]])

visualizer.predict([[1,60,36,114,70,48,60,67,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,0]])

visualizer.predict([[1,80,56,134,90,68,70,67,0.5]])

visualizer.predict([[1,80,56,134,90,68,70,67,1]])
```

```
visualizer.predict([[1,72,34,104,78,75,75,87,1]])  
visualizer.predict([[1,40,16,114,81,90,66,87,1]])  
visualizer.predict([[1,68.6,46,147,92,103,75,74,1]])  
visualizer.predict([[0,50.0,32,145,90,93,75,87,2]])  
visualizer.predict([[1,70.0,52,165,110,113,95,107,2]])  
visualizer.predict([[0,70.0,52,165,110,113,95,107,2]])  
visualizer.predict([[0,70.0,52,165,110,113,95,107,1]])  
visualizer.predict([[0,70.0,52,165,110,113,95,107,3]])  
  
import pickle  
  
filename = 'saved_model.sv'  
  
pickle.dump(visualizer, open(filename, 'wb'))  
  
#loading the saved model  
  
loaded_model = pickle.load(open('saved_model.sv', 'rb'))  
  
#Decision Tree algorithm  
  
from sklearn.tree import DecisionTreeRegressor  
  
model2 = DecisionTreeRegressor()  
  
model2.fit(X_train, y_train)  
  
from sklearn.metrics import  
mean_absolute_error,mean_squared_error,explained_variance_score,r2_score  
  
y_train_pred = model2.predict(X_train)  
  
  
print(r2_score(y_train,y_train_pred))
```

```
print(mean_absolute_error(y_train,y_train_pred))

print(mean_squared_error(y_train, y_train_pred))

y_test_pred = model2.predict(X_test)

print(r2_score(y_test, y_test_pred))

model2.predict([[0,70.0,52,165,110,113,95,107,3]])

import pickle

filename = 'model.pkl'

pickle.dump(model2, open(filename, 'wb'))

#loading the saved model

loaded_model = pickle.load(open('model.pkl', 'rb'))

#plotting the observed and the predicted Remifentanil Flow

import matplotlib.pyplot as plt

#setting the boundaries and the parameters

plt.rcParams['figure.figsize']=(16,6)

x_ax = range(len(X_test))

#plotting

plt.plot(x_ax, y_test,label ='Observed', color ='k', linestyle ='-')

plt.plot(x_ax,y_test_pred,label = 'Predicted', color ='k', linestyle ='--')

plt.ylabel('Remifentanil Flow')

plt.xlabel('Testing sample data')

plt.legend(bbox_to_anchor = (0.5,-0.2),loc = 'lower center', ncol =2, frameon = False)
```

```
plt.show()

from yellowbrick.regressor import PredictionError

visualizer2 = PredictionError(model2)

visualizer2.fit(X_train, y_train)

visualizer2.score(X_test, y_test)

visualizer2.poof()

from yellowbrick.regressor import ResidualsPlot

visualizer = ResidualsPlot(model2)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

visualizer2.predict([[0,70.0,52,165,110,113,95,107,3]])

visualizer.predict([[0,50.0,32,146,89,94,75,88,2]])

visualizer2.predict([[0,50.0,32,145,90,93,75,87,2]])

visualizer2.predict([[1,50.0,32,144,89,94,75,90,1]])

visualizer2.predict([[1,68.6,46,147,92,103,75,74,1]])

visualizer2.predict([[1,88.6,66,167,112,123,95,94,1]])

from sklearn.ensemble import GradientBoostingRegressor

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import matplotlib.pyplot as plt
```

```
from yellowbrick.regressor import PredictionError, ResidualsPlot

# Assuming X_train, X_test, y_train, y_test are already defined

# Initialize the Gradient Boosting Regressor

model2 = GradientBoostingRegressor(random_state=42) # You can adjust parameters as
needed

# Train the model

model2.fit(X_train, y_train)

# Evaluation on training set

y_train_pred = model2.predict(X_train)

print("Mean Absolute Error (Train):", mean_absolute_error(y_train, y_train_pred))

print("Mean Squared Error (Train):", mean_squared_error(y_train, y_train_pred))


# Evaluation on testing set

y_test_pred = model2.predict(X_test)

print("R-squared (Test):", r2_score(y_test, y_test_pred))

# Plotting observed vs predicted

plt.rcParams['figure.figsize'] = (16, 6)

x_ax = range(len(X_test))

plt.plot(x_ax, y_test, label='Observed', color='k', linestyle='-')

plt.plot(x_ax, y_test_pred, label='Predicted', color='k', linestyle='--')

plt.ylabel('Remifentanil Flow')

plt.xlabel('Testing sample data')
```



```
plt.legend(bbox_to_anchor=(0.5, -0.2), loc='lower center', ncol=2, frameon=False)
```

```
plt.show()
```

```
# Yellowbrick Prediction Error Visualizer
```

```
visualizer2 = PredictionError(model2)
```

```
visualizer2.fit(X_train, y_train)
```

```
visualizer2.score(X_test, y_test)
```

```
visualizer2.poof()
```

```
# Yellowbrick Residuals Plot
```

```
visualizer = ResidualsPlot(model2)
```

```
visualizer.fit(X_train, y_train)
```

```
visualizer.score(X_test, y_test)
```

```
visualizer.poof()
```

```
# Making a prediction using the trained model
```

```
prediction = model2.predict([[0, 70.0, 52, 165, 110, 113, 95, 107, 3]])
```

```
print("Predicted Remifentanil Flow:", prediction)
```